

CMSE 801 - Day 16



Creating effective data visualizations

October 28, 2024

General Course Announcements

- Quiz 3 is **today** during the last 30-minutes of class. There are extra practice notebooks on the website.
- Project Proposals were due **today**.
 - We'll use the first part of class to share your proposals with your group.
- Homework 4 should come out on Wednesday.

Questions/comments from Day 16 PCA

- Coming up with what story to tell with your data visualization can be challenging.
 - The plot should do more than just represent a basic statistic or something that could easily just be shown with a small table.
 - The visualization to create opportunities for insights not capture by the data alone
 - Figuring out the "right" way to plot your data can be hard!
- Seaborn can be a bit tricky to use as first, just like any new package, it takes time to digest the documentation and learn to use it.
 - When you get stuck trying to use a new tool, as always, googling the issue you're running into can be a good way to find examples that do something similar to what you're trying to do.
 - Seaborn can also be used as a simple way of changing the style of basic matplotlib plots. **We'll look at an example.**

It's important to think critically when we're presented with data visualizations

- Make sure to ask yourself:
 - What story is this visualization trying to tell?
 - What might this data visualization be hiding?
 - What do I need to know about the data in order to more deeply understand the visualization?

In order to determine the best way to visualize your data, you need to spend time exploring it!

- Try visualizing the data in more than one way.
- Think about whether or not you're interested in the relationships between features in your data, how the data varies with time, or something else!
 - Perhaps you need to use one or more features to create a new feature that is derived from the original ones.
- Explore the types of visualizations and think about which ones might be the right choice for the story you're trying to tell.
 - The [Data Visualisation Catalogue](#) is a great tool for this. *Try the "Search by Function" feature.*

Let's look at a some ways we can use seaborn

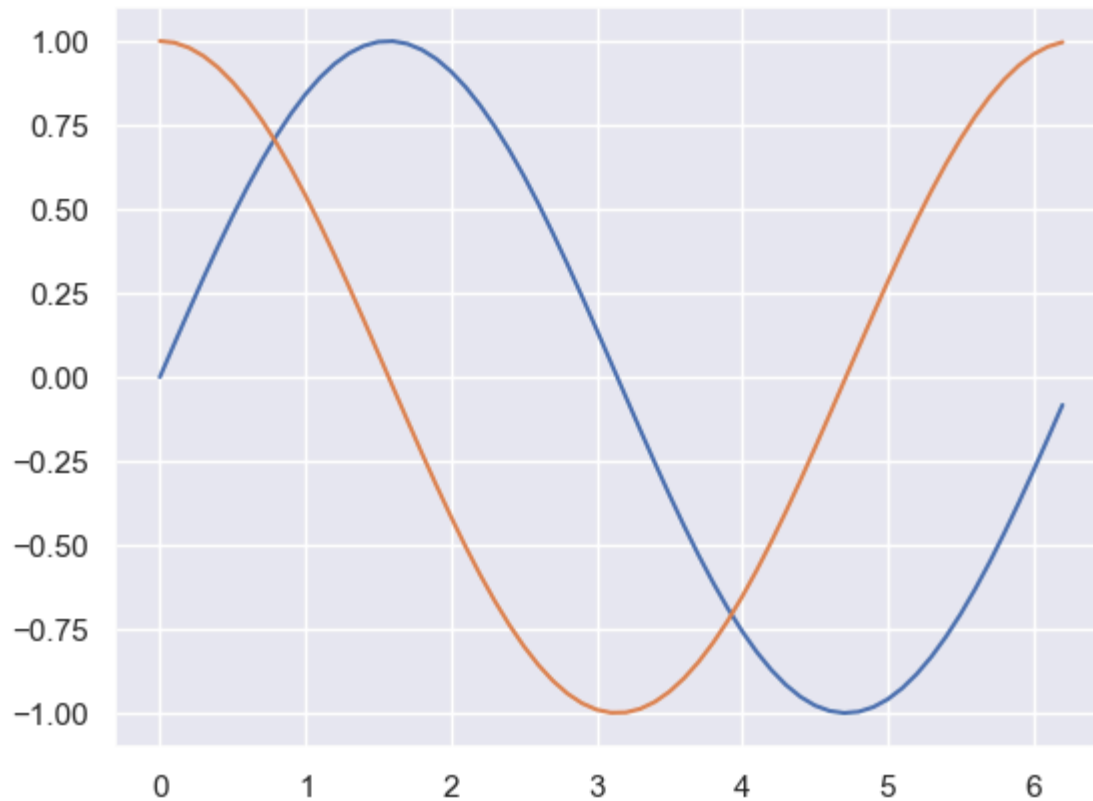
```
In [1]: # Some some imports out of the way  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
In [2]: # Let's make some data to plot  
# Create a list from 0 to 2pi with increments of 0.1  
x_start = 0  
x_stop = 2*np.pi  
dx = 0.1  
x = np.arange(x_start, x_stop, dx)  
  
# Calculate sine and cosine using NumPy  
sin_x = np.sin(x)  
cos_x = np.cos(x)
```

```
In [3]: # Turn on the seaborn styling
sns.set_theme()

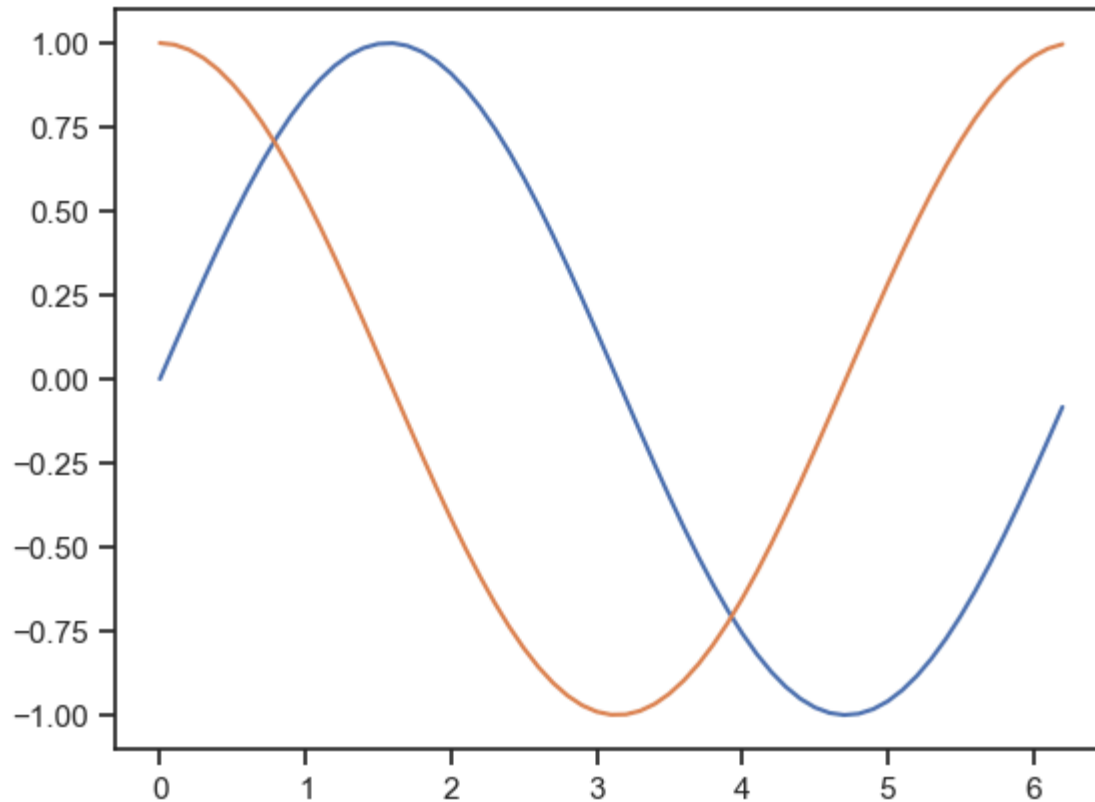
plt.plot(x,sin_x)
plt.plot(x,cos_x)
```

Out[3]: [



```
In [7]: # We can also modify the style; details here: https://seaborn.pydata.org  
sns.set_style("ticks") # we can test "ticks", "dark", "whitegrid" as well  
  
plt.plot(x,sin_x)  
plt.plot(x,cos_x)
```

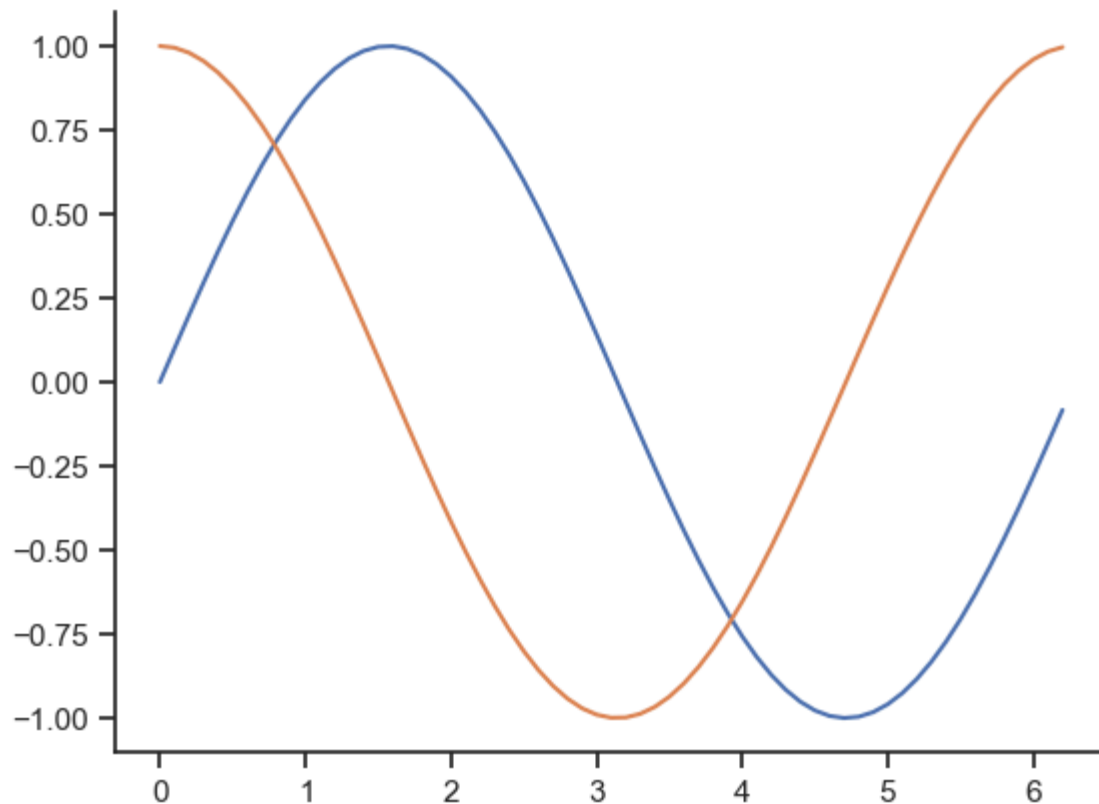
Out[7]: [`<matplotlib.lines.Line2D at 0x14dc115e0>`]



```
In [8]: # We can also modify the style; details here: https://seaborn.pydata.org
sns.set_style("ticks")

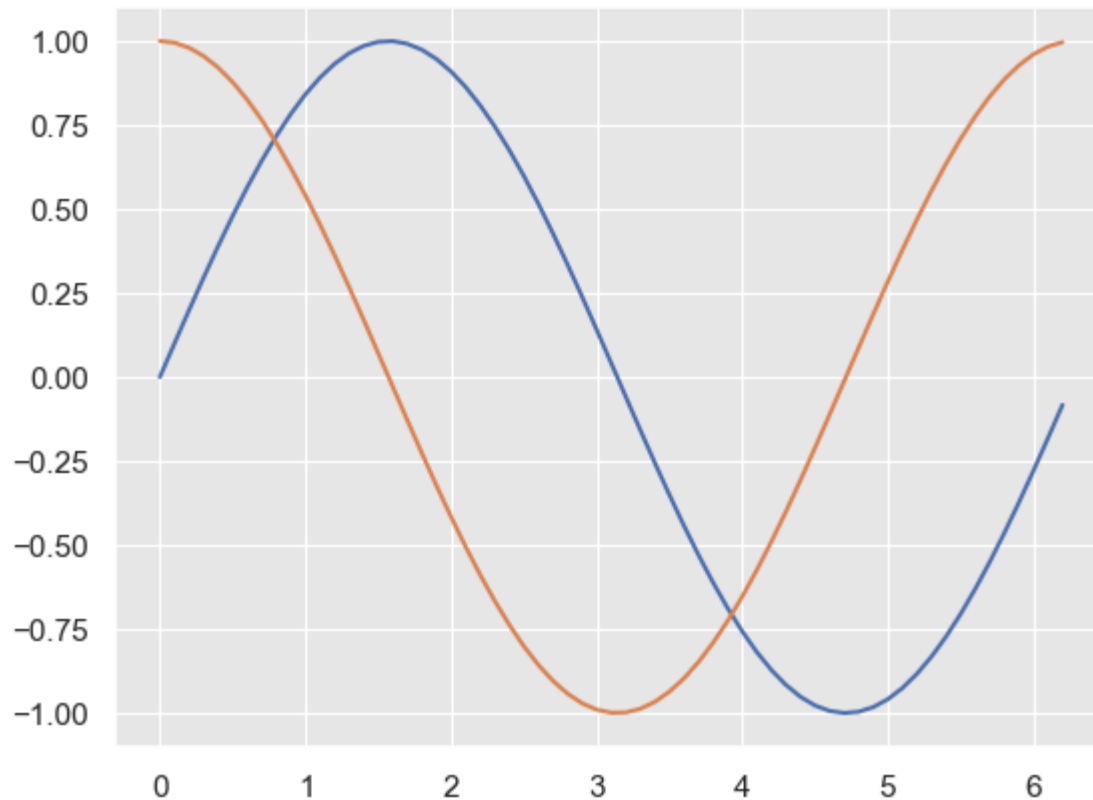
plt.plot(x,sin_x)
plt.plot(x,cos_x)

# we can remove the "spines" as well
sns.despine()
```



```
In [9]: # We can start getting really specific as well  
sns.set_style("darkgrid", {"axes.facecolor": ".9"})  
  
plt.plot(x, sin_x)  
plt.plot(x, cos_x)
```

Out[9]: [`<matplotlib.lines.Line2D at 0x14dcfe330>`]



```
In [10]: # We can see all the things we can tweak with this command:  
sns.axes_style()
```

Out[10]:

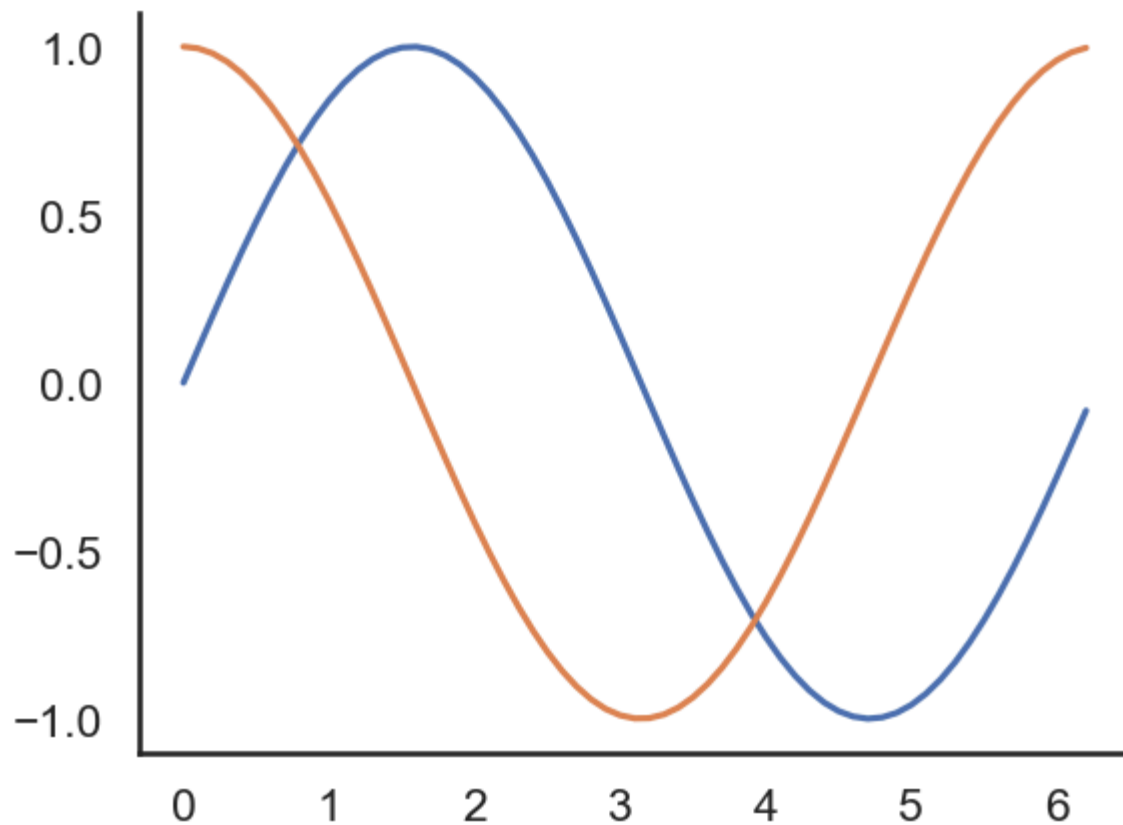
```
{'axes.facecolor': '.9',  
 'axes.edgecolor': 'white',  
 'axes.grid': True,  
 'axes.axisbelow': True,  
 'axes.labelcolor': '.15',  
 'figure.facecolor': 'white',  
 'grid.color': 'white',  
 'grid.linestyle': '-',  
 'text.color': '.15',  
 'xtick.color': '.15',  
 'ytick.color': '.15',  
 'xtick.direction': 'out',  
 'ytick.direction': 'out',  
 'lines.solid_capstyle': <CapStyle.round: 'round'>,  
 'patch.edgecolor': 'w',  
 'patch.force_edgecolor': True,  
 'image.cmap': 'rocket',  
 'font.family': ['sans-serif'],  
 'font.sans-serif': ['Arial',  
 'DejaVu Sans',  
 'Liberation Sans',  
 'Bitstream Vera Sans',  
 'sans-serif'],  
 'xtick.bottom': False,  
 'xtick.top': False,  
 'ytick.left': False,  
 'ytick.right': False,  
 'axes.spines.left': True,  
 'axes.spines.bottom': True,  
 'axes.spines.right': True,  
 'axes.spines.top': True}
```

```
In [15]: # And finally, we can set a "context" for our plot
sns.set_context("talk")

sns.set_style("white")

plt.plot(x, sin_x)
plt.plot(x, cos_x)

sns.despine()
```



But that's just the beginning!

There are entirely new style of plots that available in Seaborn and you are encouraged to look at the [tutorial page](#) to explore how these can be used.

Project proposals

- Take the next ~12 minutes to take turns presenting your project proposals to your group
 - Try to keep track of time and limit yourselves to ~3 minutes per person
 - You're encouraged to make time for getting some thoughts/feedback from your peers.

Goals for today

- Briefly present the plot you came up with in the PCA to your groups and get some immediate feedback
- Try to improve your plot based on the feedback and the information in the ICA -- **make sure you consider the provided plot criteria!**
 - If there's time, you can present your updated plot to your group.

Looking forward - Day 17 PCA: Working with 2D NumPy arrays

- You'll explore how information can be stored in two-dimensional (2D) numpy arrays.
 - These are similar, in ways, to Pandas dataframes and information can be accessed in ways similar to the ":" notation used in `.iloc`
- You'll explore how to manipulate 2D arrays and how images can be stored as a collection of 2D arrays.

